

Development Individual Project: Coding Output and Evidence of Testing

GitHub Repository

The full implementation and supporting artefacts are available at:

<https://github.com/va-angelier/va-angelier.github.io>

Relevant components:

- Unit 9 (API implementation):
SSDC_POCM7E_Assignment/Unit_9/Artifacts/API
 - Unit 11 (UI and **evidence**): SSDC_POCM7E_Assignment/Unit_11
-

1. Introduction

This project presents the implementation of a secure online retailer API and CLI prototype developed in Python using Flask. The system extends beyond a basic CRUD application by incorporating secure account management, authentication, session handling, and role-based access control. The implementation builds upon the design established in Unit 6 and represents the practical application of secure software development principles within a controlled environment.

The primary objective of the project is not only to provide functional API endpoints but to demonstrate how security controls can be embedded throughout the software development lifecycle. This aligns with the principle that security should be integrated into system design rather than applied retrospectively (Olmsted, 2024).

The project is structured across multiple units to reflect an iterative development process. Unit 9 contains the core API implementation, while Unit 11 provides supporting artefacts, including a lightweight user interface and evidence of execution and testing. This structure reflects real-world development practices, where systems evolve incrementally and are validated through continuous testing.

2. System Design Overview

The system is implemented as a RESTful API using Flask, complemented by command-line interface (CLI) functionality for administrative and testing purposes.

The API supports user account management, authentication, and CRUD operations on records.

The architecture follows a client-server model in which all security controls are enforced server-side. The frontend UI, developed in Unit 11, serves purely as a testing and demonstration layer and does not perform any security-critical validation. This ensures that trust is not placed in the client layer.

The system maintains several in-memory data structures, including users, records, active sessions, and security events. While this simplifies deployment, it also allows a clear focus on security mechanisms such as validation, authentication, and monitoring.

From a design perspective, the implementation reflects a simplified secure software development lifecycle, where design, implementation, and testing are closely integrated. The decomposition of functionality into discrete endpoints aligns with modular design principles and supports maintainability.

3. Security Design Decisions

Security was a central consideration throughout the implementation. The system incorporates multiple layers of defence to mitigate common application-level vulnerabilities, including those identified by OWASP such as injection, broken access control, and insufficient logging.

A key design decision was the implementation of strict input validation. All incoming requests must be valid JSON objects, and only expected fields are accepted. Any unexpected or malformed input is rejected with a controlled error response. This reduces the risk of injection-style attacks and enforces predictable behaviour.

In addition to structural validation, the system performs content validation, including length restrictions and pattern-based detection of suspicious input (e.g. SQL injection indicators or script tags). While simplified, this demonstrates proactive input filtering.

Authentication is implemented using a combination of password-based login and session tokens. Passwords are securely hashed using PBKDF2-HMAC-SHA256 with a unique salt per user, ensuring resistance against brute-force and rainbow table attacks. Session tokens are generated using cryptographically secure random values and must be included in subsequent requests.

Authorisation is enforced through role-based access control. Users are assigned roles (e.g. user or admin), and certain operations, such as deleting users or accessing security logs, require elevated privileges. This mitigates the risk of unauthorised access to sensitive functionality.

To further enhance security, the system implements account lockout after repeated failed login attempts. This reduces the effectiveness of brute-force attacks and demonstrates adaptive security behaviour.

The system also includes structured security event logging. Events such as authentication attempts, authorisation failures, and administrative actions are recorded with timestamps and contextual information. This supports monitoring and aligns with the principle that insufficient logging is itself a security weakness.

Finally, a configurable “secure mode” allows security controls to be enabled or disabled at runtime. This feature supports testing and demonstrates the impact of security configurations on system behaviour.

4. Implementation

The API was implemented in Python using Flask, with a modular structure separating configuration, validation, authentication, and endpoint logic.

The system includes endpoints for user registration, login, session validation, logout, and profile retrieval. These endpoints ensure secure handling of user credentials and session state. Passwords are never stored in plaintext, and verification is performed using secure hash comparison.

CRUD functionality is implemented through dedicated endpoints for creating, retrieving, updating, and deleting records. These operations are protected by both API key validation and session-based authentication, ensuring that only authorised users can modify data.

Administrative endpoints extend the functionality further, allowing user deletion and retrieval of security events. These endpoints require both a valid session and appropriate role permissions, demonstrating layered access control.

All responses are returned as structured JSON, with consistent formatting across both success and error cases. Error messages are deliberately controlled to avoid exposing sensitive internal information.

The code adheres to PEP-8 style guidelines (van Rossum, 2001), ensuring readability and maintainability. Inline comments are used to document key logic and highlight security-related decisions.

5. Testing Strategy and Evidence

Testing was conducted using a combination of automated and manual methods to ensure both functional correctness and security robustness.

Automated testing was performed using pytest. Unit tests were designed to verify core functionality, including user registration, authentication, and CRUD operations. These tests confirm that the system behaves correctly under expected conditions.

Static analysis was conducted using a linter to enforce coding standards and identify potential issues. The linter output demonstrates compliance with PEP-8 guidelines and the absence of critical warnings.

Manual testing was used to evaluate the system's behaviour under adverse conditions. This included testing malformed JSON input, missing fields, incorrect data types, invalid session tokens, and unauthorised access attempts. These scenarios confirm that the system correctly rejects invalid requests and enforces security controls.

This multi-layered testing approach ensures that both functionality and security mechanisms are validated, providing confidence in the reliability of the system.

6. Evidence of Execution

The functionality of the system was validated through a series of execution tests using HTTP requests. These tests demonstrate correct operation of both user management and record management endpoints.

All evidence, including full-resolution screenshots and execution logs, is available in the GitHub repository under [Unit_11/Evidence](#).

7. Critical Evaluation

While the implementation demonstrates strong alignment with secure software development principles, several limitations remain.

The use of in-memory storage limits the system's applicability in production environments. Persistent storage with appropriate encryption and access controls would be required for real-world deployment.

Session management is also simplified, with no token expiry or refresh mechanisms. This could lead to security risks if tokens are compromised.

The pattern-based detection of suspicious input is intentionally basic and would not be sufficient to prevent advanced injection attacks. More robust approaches, such as parameterised queries and formal input sanitisation libraries, would be required.

Additionally, the system does not implement rate limiting or transport-level security such as HTTPS. These controls are essential for protecting against denial-of-service attacks and ensuring data confidentiality in transit.

Despite these limitations, the implementation effectively demonstrates how security can be embedded within application logic. The combination of validation, authentication, authorisation, and monitoring reflects a defence-in-depth approach consistent with secure software development practices (Olmsted, 2024).

8. Conclusion

This project demonstrates the practical implementation of a secure online retailer API using Python and Flask. By integrating authentication, role-based access control, input validation, and security monitoring, the system addresses key application-level security risks.

The combination of structured design, comprehensive testing, and evidence-based validation reflects a mature approach to secure software development. While simplified, the implementation provides a strong foundation for further enhancement and highlights the importance of integrating security throughout the software development lifecycle.

References

Olmsted, A. (2024) *Security-Driven Software Development*. Birmingham: Packt Publishing.

OWASP (2021) *OWASP Top 10: A03 – Injection*. Available at:
https://owasp.org/Top10/A03_2021-Injection/

van Rossum, G. (2001) *PEP 8 – Style Guide for Python Code*. Available at:
<https://peps.python.org/pep-0008/>